11 December 2025 | M2 SAR | Paris, France

Verification of timed systems

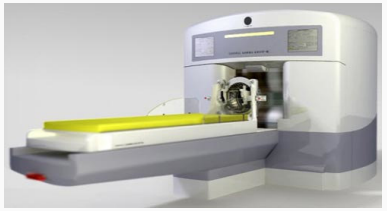
M2 SAR - UE Applications et systèmes temps réel répartis embarqués

Dylan Marinho

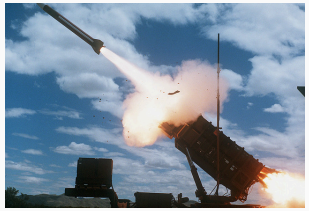
Sorbonne Université, CNRS UMR 7606, LIP6

Context: Verifying complex timed systems

- **Critical** systems: Failures may result in **dramatic** consequences
- Need for early bug detection
  - Bugs discovered when final testing: **expensive**
  - Need for a thorough **specification** and **verification** phase



Therac-25
(USA, 1980s)



MIM-104 Pat. Mis. Fail.
(Iraq, 1991)

Sleipner A offshore platform
(Norway, 1991)

Ariane flight V88
(France, 1996)

The Therac-25 case study

- Radiation therapy machine used in the 1980s
- Involved in accidents between 1985 and 1987, in which patients were given **massive overdoses of radiation**
  - Approximately **100 times** the intended dose!
  - Numerous causes, including **race condition**

> The failure only occurred when a particular nonstandard sequence of keystrokes was entered on the VT-100 terminal which controlled the PDP- 11 computer: an `X` to (erroneously) select 25MV photon mode followed by ↑, `E` to (correctly) select 25 MeV Electron mode, then `Enter`, all **within eight seconds**.

Why timed automata?

- **Formal** models for timed critical systems specification
- Use **model checking** to verify their **timed properties**
  - Properties expressed in **MITL** and **TCTL** logics

Qualitative

Finite automata give a **simple syntax** and a **formal context**:

- executions, sequences of actions
- modular definition (composition, synchronization, etc.)
- powerful checking algorithms

Quantitative

We cannot include quantitative aspects in finite automata

- ⏳ Time

  "the airbag always eventually inflates after a crash", but maybe 10 seconds after the crash

- 🌡️ Temperature

  "the alarm always eventually rings after the temperature is high", but maybe when the temperature is above 200 degrees"

Timed formalisms

A number of formalisms have been proposed to model timed systems, including:

- Timed automata [AD94]
- Timed Petri nets [Mer74]
- Timed process algebras [Sun+09]
- ...

We use **timed automata** in this course

See [Bér+05][Srb08][Bér+13]for a comparison between timed Petri nets and timed automata

---

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

[Mer74] Philip Meir Merlin, "A study of the recoverability of computing systems," Doctoral dissertation, 1974.

[Sun+09] Jun Sun, Yang Liu, Jin Song Dong, and Xian Zhang, "Verifying Stateful Timed CSP Using Implicit Clocks and Zone Abstraction," in *ICFEM 2009*, 2009.

[Bér+05] Béatrice Bérard, Franck Cassez, Serge Haddad, Didier Lime, and Olivier H. Roux, "Comparison of the Expressiveness of Timed Automata and Time Petri Nets," 2005.

[Srb08] Jirí Srba, "Comparing the Expressiveness of Timed Automata and Timed Extensions of Petri Nets," in *FORMATS 2008*, 2008.

[Bér+13] Béatrice Bérard, Franck Cassez, Serge Haddad, Didier Lime, and Olivier H. Roux, "The expressive power of time Petri nets," *Theoretical Computer Science*, 2013.

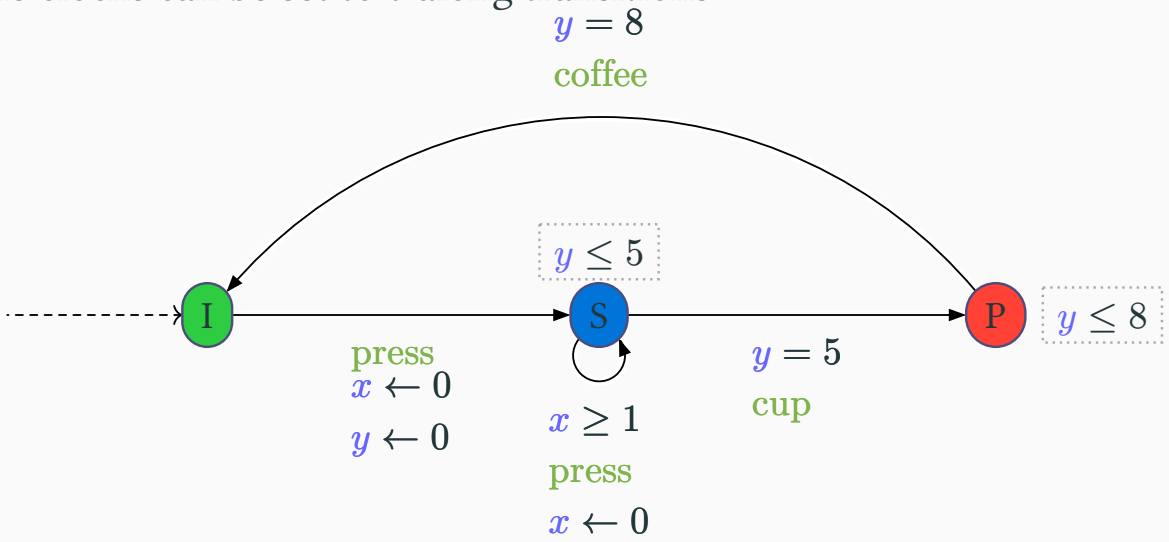
Timed automata

Syntax

Syntax overview

Timed automaton

- Finite-state automaton (sets of locations, transitions, and actions) augmented with a set X of clocks [AD94]
  - Real-valued variables evolving linearly **at the same rate**
- Features:
  - Location **invariant**: property to be verified to stay at a location
  - Transition **guard**: property to be verified to enable a transition
  - Clock **reset**: some of the clocks can be set to 0 along transitions



[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

Formal definition

Formal definition of a timed automaton

Definition 1 (Timed automaton)

A timed automaton is a tuple $\mathcal{A} = (L, \Sigma, \ell_i, F, \mathbb{X}, I, E)$ where:

- L is a set of states (called **locations**)
- Σ is a set of actions
- ℓ_i is an initial location
- $F \subseteq L$ is a set of final states
- $\mathbb{X}$ is a set of clocks
- I is an invariant function, assigning to each location $\ell \in L$ a constraint $I(\ell)$ over $\mathbb{X}$
- E is a set of edges of the form (ℓ, g, a, R, ℓ') where:
  - ℓ, ℓ' are the source and target locations
  - g is a constraint over $\mathbb{X}$
  - $a \in \Sigma$
  - $R \subseteq \mathbb{X}$ is a set of clocks to reset

♣

Clock constraint

Definition 2 (Clock constraint)

A clock constraint is a conjunction of atomic constraints

♣

What is an atomic constraint?

- Various definitions in the literature:
  - Originally [AD94]: $x \in [c_1, c_2]$ with $c_1 \in \mathbb{N}$ and $c_2 \in \mathbb{N}_\infty$
  - Comparing clock values (**diagonal constraints**): $x_1 - x_2 \sim c$ with $\sim \in \{<, \leq, =, \geq, >\}$

For now, we assume the following syntax:

- $x \sim c$, with $c \in \mathbb{Q}$ *as an classical extension of* $c \in \mathbb{N}$

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

Exercise: Drawing a TA

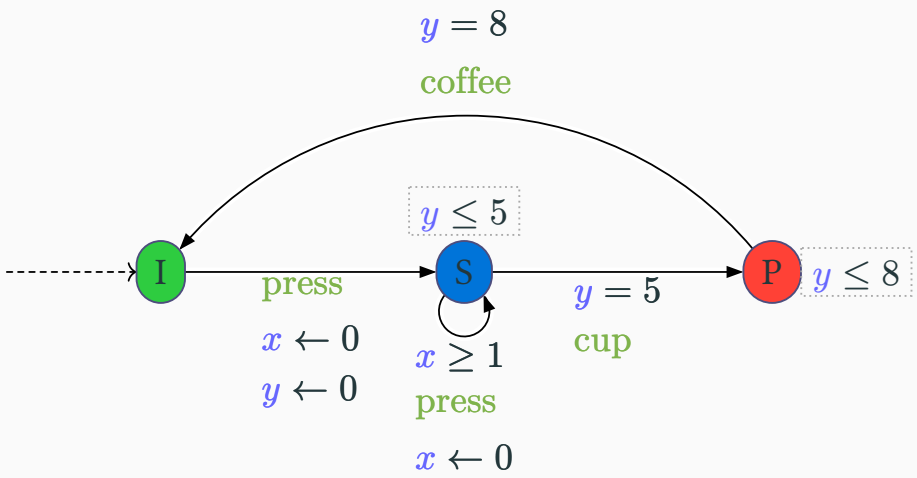
Draw the TA such that:

- $L = \{\ell_1, \ell_2, \ell_3, \ell_4\}$
- $\ell_\mathrm{i} = \ell_1$
- $F = \{\ell_2, \ell_4\}$
- $\Sigma = \{a, b, c\}$
- $\mathbb{X} = \{x_1, x_2\}$
- $I(\ell_1) = x_1 \leq 3$,
  $I(\ell_3) = x_2 \geq 2$
- $$E = \left\{ \begin{array}{l} (\ell_1, x_1 \geq 2, a, \{x_1\}, \ell_2) \\ (\ell_1, x_2 \leq 1, b, \emptyset, \ell_3) \\ (\ell_2, x_2 = 1, c, \{x_2\}, \ell_2) \\ (\ell_2, \top, a, \emptyset, \ell_3) \\ (\ell_3, \top, b, \{x_1, x_2\}, \ell_4) \\ (\ell_4, x_2 > 2, c, \emptyset, \ell_3) \end{array} \right\}$$

- Are all the locations reachable?
- Can all transitions be taken?
- What is the minimal time before taking the transition from ℓ_4 to ℓ_3?

Exercise: The coffee machine

Give the formal TA corresponding to the timed coffee machine.



Parallel composition

Parallel composition of timed automata

As for finite automata, we can compose timed automata through **parallel composition**.

From $\mathcal{A}_1 = \left(L_1, \Sigma_1, \ell_{\mathrm{i}_1}, F_1, \mathbb{X}_1, I_1, E_1\right)$ and $\mathcal{A}_2 = \left(L_2, \Sigma_2, \ell_{\mathrm{i}_2}, F_2, \mathbb{X}_2, I_2, E_2\right)$

we can define $\mathcal{A}_1 \parallel \mathcal{A}_2$ as $\mathcal{A} = (L, \Sigma, \ell_{\mathrm{i}}, F, \mathbb{X}, I, E)$ with:

- $L = L_1 \times L_2$
- $\Sigma = \Sigma_1 \cup \Sigma_2$
- $\ell_{\mathrm{i}} = \left(\ell_{\mathrm{i}_1}, \ell_{\mathrm{i}_2}\right)$
- $F = F_1 \times L_2 \cup L_1 \times F_2$
- $\mathbb{X} = \mathbb{X}_1 \cup \mathbb{X}_2$
- $I((\ell_1, \ell_2)) = I_1(\ell_1) \wedge I_2(\ell_2)$

Parallel composition of timed automata

- $((\ell_1, \ell_2), g, a, R, (\ell_1', \ell_2')) \in E$ if:
  - $a \in \Sigma_1 \cap \Sigma_2$ and $\exists g_1, g_2, R_1, R_2$ *s.t.*
    - $(\ell_1, g_1, a, R_1, \ell_1') \in E_1$ and $(\ell_2, g_2, a, R_2, \ell_2') \in E_2$,
    - $g = g_1 \wedge g_2$
    - $R = R_1 \cup R_2$

  } We advance in both automata (respecting the guards and performing the resets)

  - or, $a \in \Sigma_1 \setminus \Sigma_2$ and $(\ell_1, g, a, R, \ell_1') \in E_1$ and $\ell_2 = \ell_2'$

  } We advance only in the first automaton

  - or, $a \in \Sigma_2 \setminus \Sigma_1$ and $(\ell_2, g, a, R, \ell_2') \in E_2$ and $\ell_1 = \ell_1'$

  } We advance only in the second automaton

Semantics

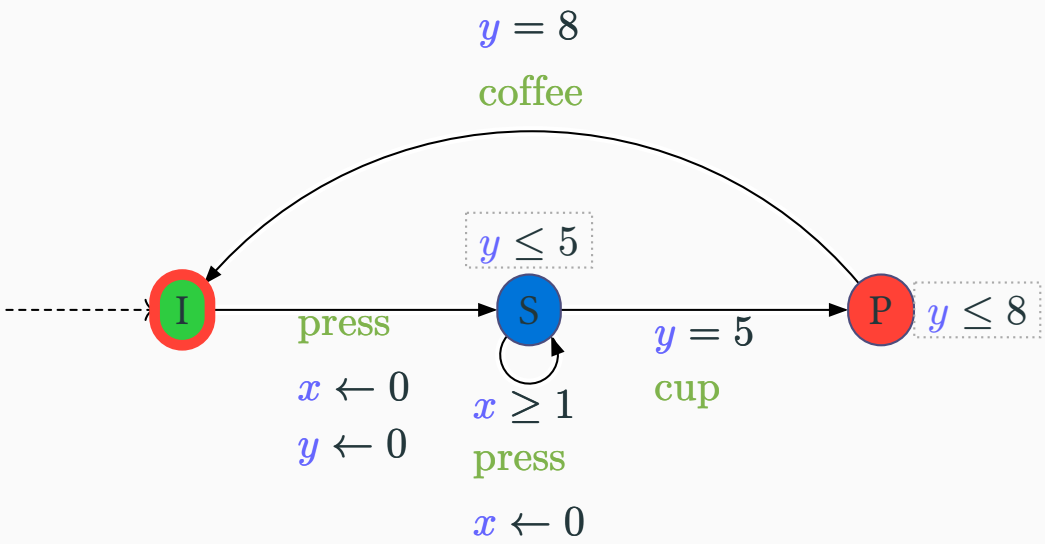
Concrete semantics

Concrete semantics

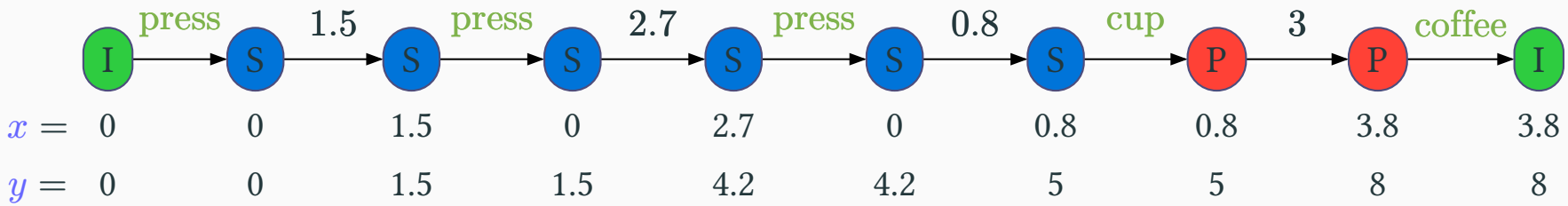
- **Concrete state**: pair (ℓ, μ) where:
  - $\ell \in L$ is a location
  - $\mu : \mathbb{X} \to \mathbb{R}^+$ assigns a non-negative real value to each clock

  e.g. $(\ell_1, \{x \mapsto 2.5, y \mapsto 0\})$
- **Concrete run**: alternating sequence of concrete states and:
  - delay transitions $\to$ time elapsing
  - discrete transitions $\to$ instantaneous change with action

Example: Concrete runs



- Coffee with two doses of sugar



Formal definition

Definition 1 (Concrete semantics of a timed automaton)

The concrete semantics of a timed automaton $\mathcal{A} = (L, \Sigma, \ell_i, F, \mathbb{X}, I, E)$ is given by a **timed transition system** $\mathcal{T}_{\mathcal{A}} = (\mathfrak{S}, \mathfrak{s}_0, \Sigma \cup \mathbb{R}^+, \to)$ where:

- $\mathfrak{S} = \{(\ell, \mu) \mid \ell \in L, \mu : \mathbb{X} \to \mathbb{R}^+ \text{ s.t. } \mu \models I(\ell)\}$
- $\mathfrak{s}_0 = (\ell_i, \mu_0)$
- $\to$ consists of:
  - **Delay transition**: $(\ell, \mu) \overset{d}{\mapsto} (\ell, \mu + d)$
    with $d \in \mathbb{R}^+$, *if* $\forall d' \in [0, d], (\ell, \mu + d') \in \mathfrak{S}$
  - **Discrete transition**: $(\ell, \mu) \overset{a}{\mapsto} (\ell', \mu')$
    if $(\ell, \mu), (\ell', \mu') \in \mathfrak{S}$ *and* $\exists (\ell, g, a, R, \ell') \in E$ *s.t.* $\mu \models g$ *and* $\mu' = [\mu]_R$ [1]

♣

[1] Notation: $[\mu]_R = \mu[R \mapsto 0] = \begin{cases} x \mapsto 0 & \text{if } x \in R \\ x \mapsto \mu(x) & \text{otherwise} \end{cases}$

Formal definition

Notation

We write $(\ell, \mu) \xrightarrow{(d,a)} (\ell', \mu')$ or $((\ell, \mu), (d, a), (\ell', \mu')) \in \rightarrow$ for a combination of a delay and a discrete transition

Issues (?) with concrete semantics

- Is $\mathfrak{T}_{\mathcal{A}}$ finite? ✗ *(infinite number of states due to continuous time)*
- Is $\mathfrak{T}_{\mathcal{A}}$ finitely branching? ✗ *(infinite choices of delays)*
- Is it a problem?

Time word, time language

Time word & Time language

- A **time word** is a sequence of pairs of:
  - an action,
  - a time delay since the previous action

Given a run

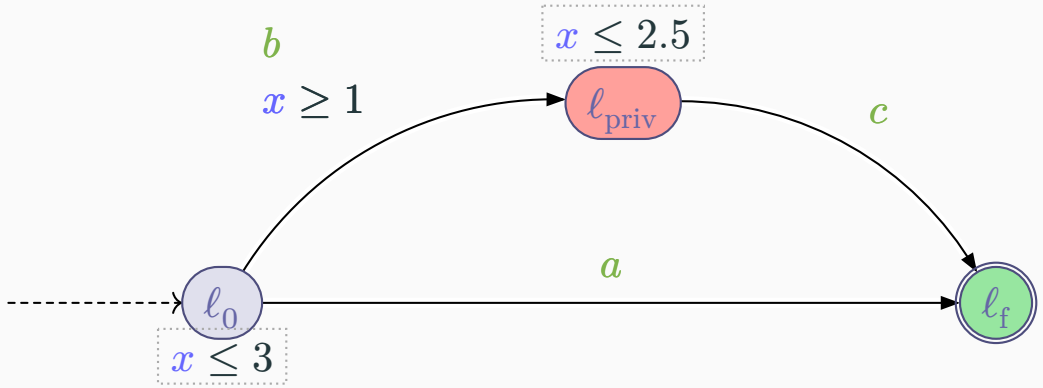
$$\left(\ell_0, \mu_0 \overset{(d_0,a_0)}{\longrightarrow} (\ell_1, \mu_1), \ldots, \overset{(d_{n-1},a_{n-1})}{\longrightarrow} (\ell_n, \mu_n)\right)$$

we can generate the timed word

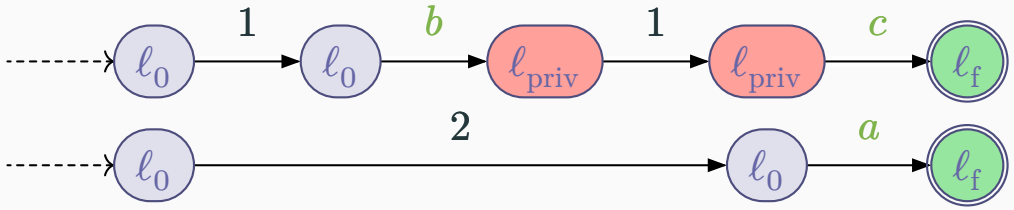
$$(a_0, d_0), (a_1, d_0 + d_1), \ldots, \left(a_{n-1}, \sum_{k=0}^{n-1} d_k\right)$$

- **Accepting run**: a run ending in a location $\ell \in F$
- **Timed language** $\mathcal{L}(\mathcal{A})$: set of time words generated by accepting runs of $\mathfrak{T}_{\mathcal{A}}$

Example: Accepting runs and timed language



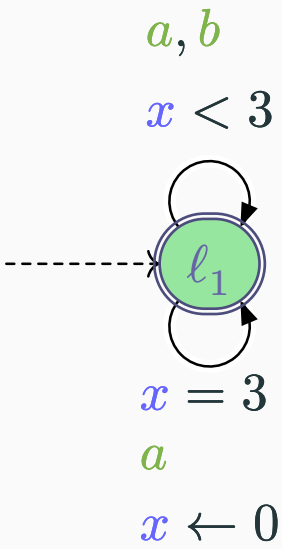
- Two examples of accepting runs:



- Timed language: $\mathcal{L}(\mathcal{A}) = \{(a,t) \mid t \in [0,3]\} \cup \{(b,t_b)(c,t_c) \mid 1 \leq t_b \leq t_c \leq 2.5\}$

Exercise: Timed language

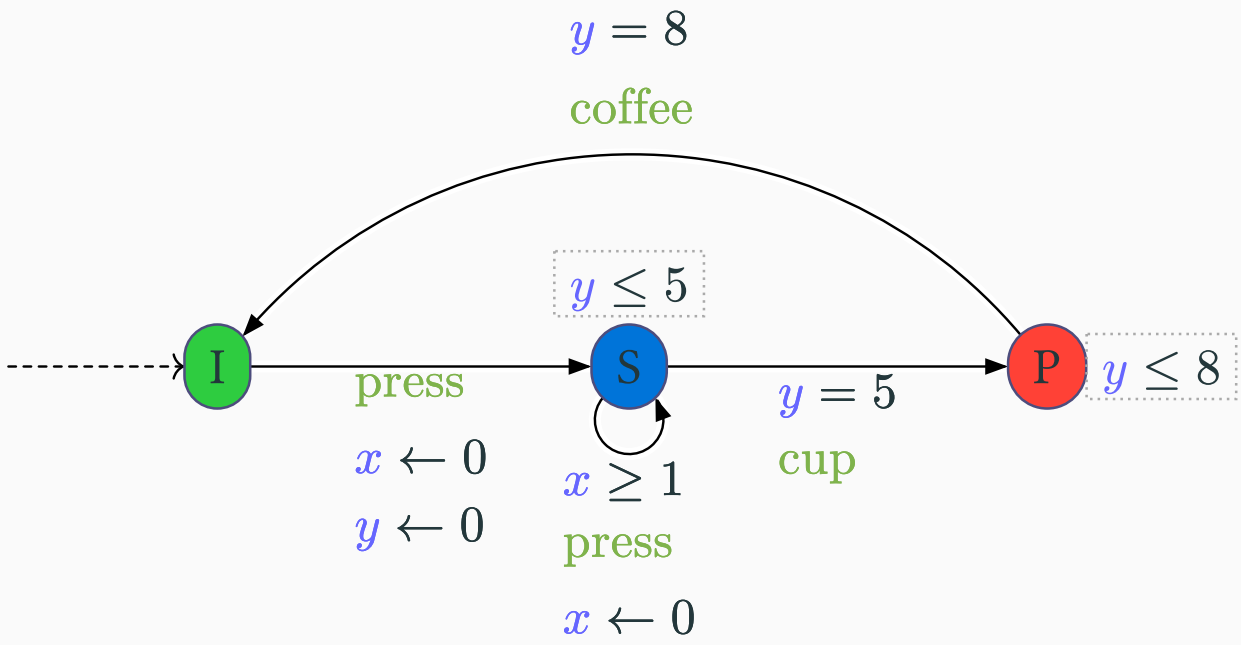
Give the timed language of the following automaton: [AD94]



[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

Exercise: Timed language (2)

Give the timed language of the following coffee machine



Accepting locations?

- We can consider automata with or without accepting locations
  - Often, timed automata with no accepting locations are called **timed safety automata** [Hen+94]
- In that case, the timed language can be defined as:
  - All possible timed words read by the automaton
  - All possible maximal timed words read by the automaton
    Maximal: infinite or that cannot be extended
  - All possible infinite timed words read by the automaton

Theorem

The expressive power of timed safety automata is **strictly less** than timed automata with accepting locations [HKW95]

[Hen+94] Thomas A. Henzinger, Xavier Nicollin, Joseph Sifakis, and Sergio Yovine, "Symbolic Model Checking for Real-Time Systems," *Information and Computation*, 1994.

[HKW95] Thomas A. Henzinger, Peter W. Kopke, and Howard Wong-Toi, "The Expressive Power of Clocks," 1995.

Specifying with timed automata

Exercise: The railroad gate controller

[AHV93]

Design three timed automata to model a railroad gate controller:

1. **Train**: once it is approaching (approach), it will come in (in) after at least 30 time units, then go out (out) and finally be far away (faraway) after at most 50 time units (after in)
2. **Gate**: once it receives the command to lower (lower), it will start to lower; upon a reception of a command to raise (raise), it will start to raise. The time to lower and to raise the gate is an interval [20, 22]
3. **Controller**: when the train is approaching, it sends the command to lower the gate within [8,10] time units; when the train is far away, it sends the command to raise the gate within [2, 4] time units

These TAs are cyclic, *i.e.* they repeat the same behavior indefinitely

---

[AHV93] Rajeev Alur, Thomas A. Henzinger, and Moshe Y. Vardi, "Parametric real-time reasoning," 1993.

Solution: The railroad gate controller

Exercise: A timed digicode

Design a timed automaton modeling a timed digicode with the following behavior:

1. The digicode accepts a fixed code of 4 digits (for instance “4650”)
2. If no digit is entered within 2 time units after the previous digit, the input is reset
3. After entering the correct code, the door remains open for 5 time units before closing
4. A reset button “R” can be pressed at any time to reset the input

The TA is cyclic, *i.e.* it repeats the same behavior indefinitely

Solution: A timed digicode

Exercise: A traffic light controller

Design a timed automaton modeling a traffic light controller for a pedestrian crossing:

1. The traffic light cycles between green, yellow and red for cars
2. The green light lasts between 30 and 60 seconds
3. The yellow light lasts exactly 5 seconds
4. The red light lasts at most 30 seconds
5. A pedestrian can request to cross by pushing a button
6. If the button is pressed while the light is green, it will turn yellow after having been green for at least 10 seconds
7. If the button is pushed during the yellow or red light, nothing changes

The TA is cyclic, *i.e.* it repeats the same behavior indefinitely

Solution: A traffic light controller

Exercise: Paying for a coffee

Model a timed automaton for a coffee machine with the following specifications:

1. When a coin is inserted, if nothing happens within 10 seconds, it is refunded
2. A coin can be refunded if the user presses the refund button
3. When a coin is present in the machine, the user can request a coffee
4. Making a coffee takes 30 seconds. During this time, no action is possible from the user

The TA is cyclic, *i.e.* it repeats the same behavior indefinitely

Solution: Paying for a coffee

Some words about decidablity

Timed automata
Syntax
Semantics
Specifying with timed automata

Some words about decidablity
Regions
Zones

Timed temporal logics
MTL / MITL
TCTL
Observers

Open note

What is decidability?

Decidability?

Definition 1

A decision problem is **decidable** if one can design an algorithm that, for any input of the problem, can answer yes or no (in a finite time, with a finite memory).

♣

Examples:

- "given three integers, is one of them the product of the other two?"
- "given a Turing machine, will it eventually halt?"
- "given a timed automaton, does there exist a run from the initial state to a given location ℓ?"

Why study decidability?

If a decision problem is *undecidable*, it is hopeless to look for algorithms yielding exact solutions (because that is impossible)

Decidability results

Decidability results for timed automata

- **Reachability problem**: given a TA, does exist an accepting run?
  ✓ Decidable *(PSPACE-complete)* [AD94]
  - *Deciding whether the language is empty (**emptiness problem**)*
  - *Deciding whether a given location is reachable*
- **Liveness**: given a timed Büchi automaton[2], does there exist an infinite accepting run?
  ✓ Decidable [AD94]
- **Universality problem**: given a TA, is its language equal to the set of all time words?
  ✗ Undecidable [AD94]
- **Language inclusion problem**: given two TAs, is the language of the first included in the language of the second?
  ✗ Undecidable [AD94]

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

[2]TA with a Büchi acceptance condition

Decidability results for timed automata (2)

- **Language equivalence problem**: given two TAs, do they have the same language?
  ✗ Undecidable [AD94]
- **Complementability**: given a TA, does there exist a TA recognizing the complement of its language?
  ✗ Undecidable [Tri06][Fin06]

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

[Tri06] Stavros Tripakis, "Folk theorems on the determinization and minimization of timed automata," *Information Processing Letters*, 2006.

[Fin06] Olivier Finkel, "Undecidable Problems About Timed Automata," in *FORMATS 2006*, 2006.

Regions

The time is dense

- Transitions can be taken anytime
  - **Infinite** number of concrete states
  - **Infinite** number of runs
  - But, model checking needs a **finite** structure

The challenge

Some runs are **equivalent** – can we group them to obtain a finite structure?

The idea

Abstract the values of the clocks using:

- **Region** automaton [AD94]
- **Zone** automaton [BY03]

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.
[BY03] Johan Bengtsson and Wang Yi, "Timed Automata: Semantics, Algorithms and Tools," 2003.

Region automaton - The intuition

Given two clock valuations, the exact value of clocks **does not matter**, as long as:

- 📊 the **integer** parts of the clocks are the same
- ⚖️ the relative order of the **fractional** parts of the clocks are the same

---

- ⬆️ or both clocks **exceed** the largest constant of the TA

Important note ⚠️

We classically extend the original definition of TA to allow **rational** numbers in guards and invariants. The construction of regions supposes to have **only integer constants**.

It can be adapted to rational constants by multiplying all constants in the TA by the least common multiple of their denominators

Region equivalence

Region equivalence

Let c_i denote the maximal constant with which clock x_i is compared in the TA

Definition 1 (Region equivalence [AD94])

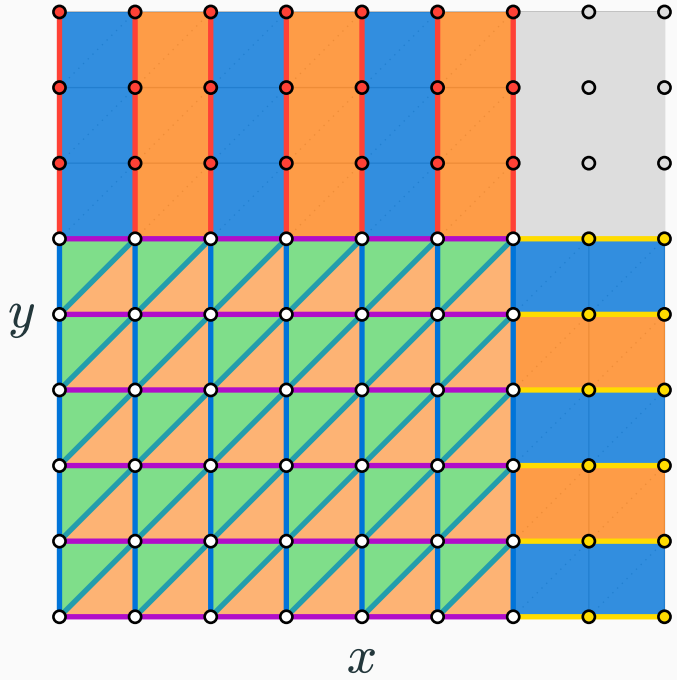
Two clock valuations μ and μ' are **equivalent**, denoted $\mu \approx \mu'$, if for all $x_i, x_j \in \mathbb{X}$

- $\lfloor \mu(x_i) \rfloor = \lfloor \mu'(x_i) \rfloor$ or $\mu(x_i) > c_i$ and $\mu'(x_i) > c_i$
- if $\mu(x_i) \leq c_i$ and $\mu(x_j) \leq c_j$, then $\mathrm{fr}(\mu(x_i)) \leq \mathrm{fr}(\mu(x_j))$ iff $\mathrm{fr}(\mu'(x_i)) \leq \mathrm{fr}(\mu'(x_j))$
- if $\mu(x_i) \leq c_i$, then $\mathrm{fr}(\mu(x_i)) = 0$ iff $\mathrm{fr}(\mu'(x_i)) = 0$

♣

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

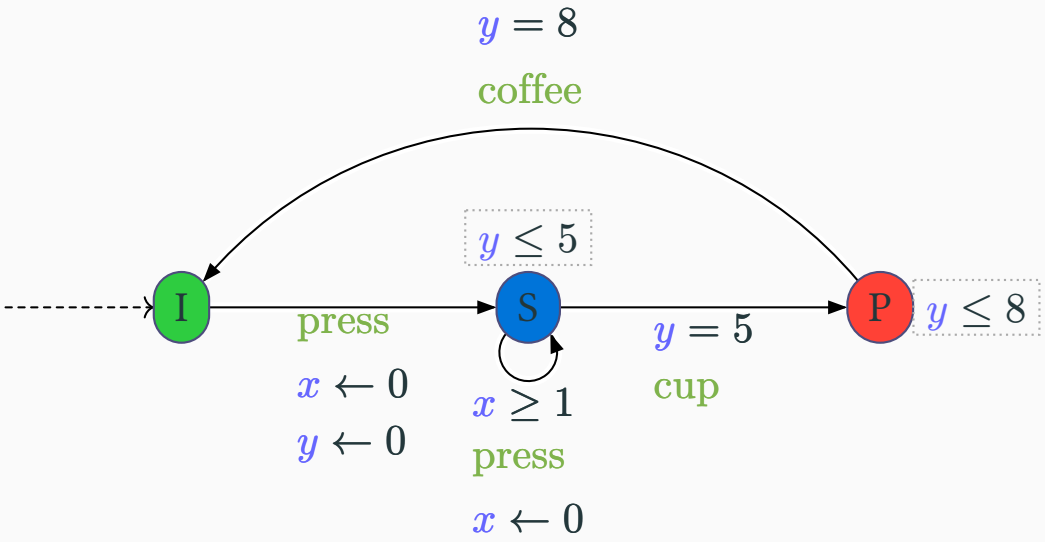
Graphical representation of region equivalence

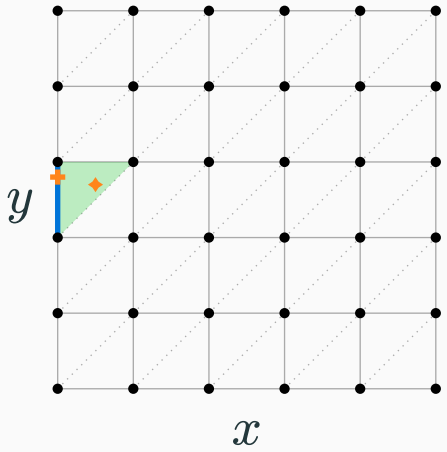


The region graph in the plane (x, y) is composed as follows:

- points (n, m) with $n \leq c_x$ and $m \leq c_y$
- lines before c_x and c_y
- triangles before c_x and c_y
- rectangles and lines when one clock exceeds its maximal constant

Example: Region equivalence





- $\mu_1 = \begin{cases} x \mapsto 0.5 \\ y \mapsto 2.7 \end{cases}$
- $\mu_2 = \begin{cases} x \mapsto 0.2 \\ y \mapsto 2.8 \end{cases}$

- $\mu_3 = \begin{cases} x \mapsto 1.3 \\ y \mapsto 0.7 \end{cases}$
- $\mu_4 = \begin{cases} x \mapsto 3.9 \\ y \mapsto 0.2 \end{cases}$

- $\mu_1 = \begin{cases} x \mapsto 0.5 \\ y \mapsto 2.7 \end{cases}$
- $\mu_5 = \begin{cases} x \mapsto 0.8 \\ y \mapsto 2.7 \end{cases}$

- $\mu_1 = \begin{cases} x \mapsto 0.5 \\ y \mapsto 2.7 \end{cases}$
- $\mu_6 = \begin{cases} x \mapsto 0 \\ y \mapsto 2.8 \end{cases}$

Region graph

Region graph

Definition 2 (Region graph)

Let $\mathcal{A}$ be a timed automaton. The **region graph** of $\mathcal{A}$ is the transition system $\mathcal{RG}_{\mathcal{A}} = (\mathcal{S}_{\text{RG}}, s_0, \Sigma, \rightsquigarrow)$ where:

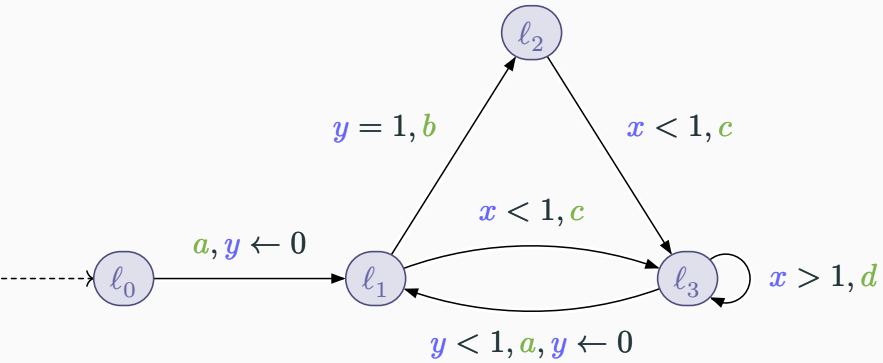
- $\mathcal{S}_{\text{RG}} = \{(\ell, r) \mid \ell \in L, r \in \mathcal{R}_{\mathcal{A}}\}$
- $s_0 = (\ell_{\text{i}}, r_0)$ with r_0 the region containing the initial valuation $\vec{0}$
- $((\ell, r), a, (\ell', r')) \in \rightsquigarrow$ iff there exist $\mu \in r$ and $\mu' \in r'$ such that $((\ell, \mu), a, (\ell', \mu')) \in \rightarrow$[1]

♣

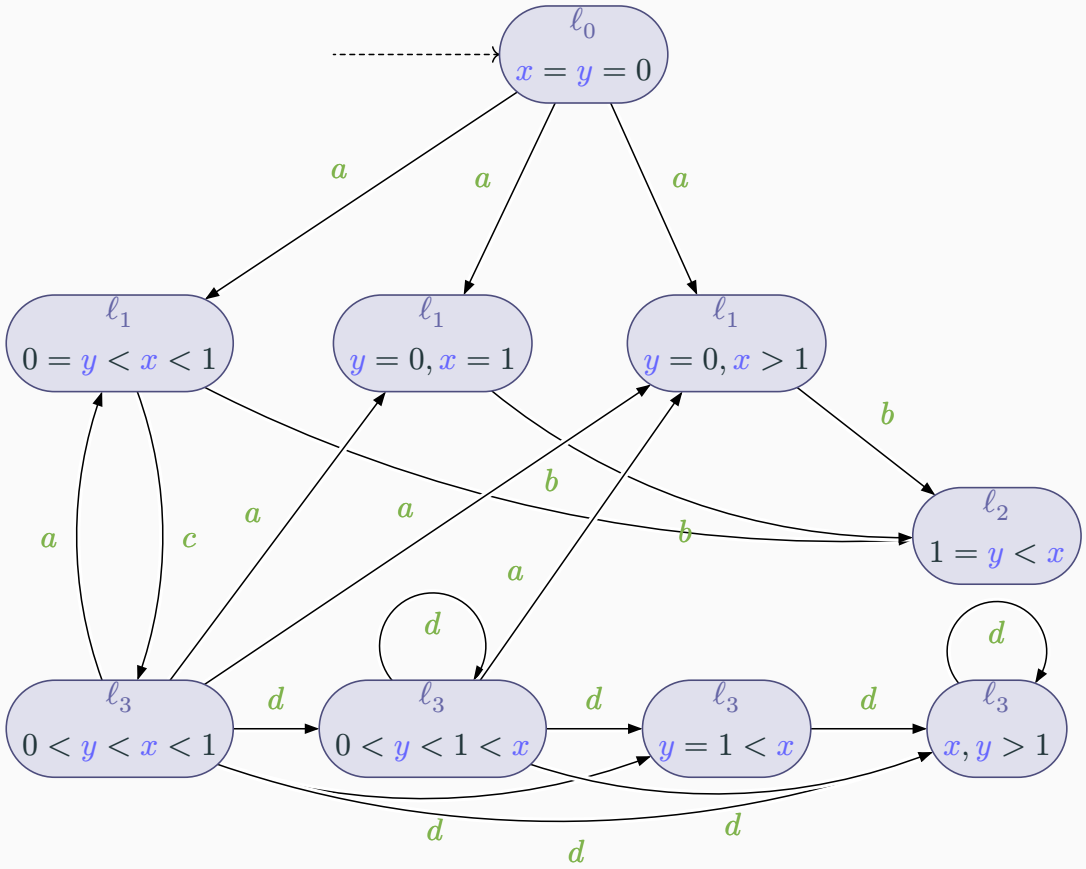
[1] $\rightarrow$ is the transition relation of the concrete semantics of $\mathcal{A}$

Region graph construction

[Bou08], [AD94]

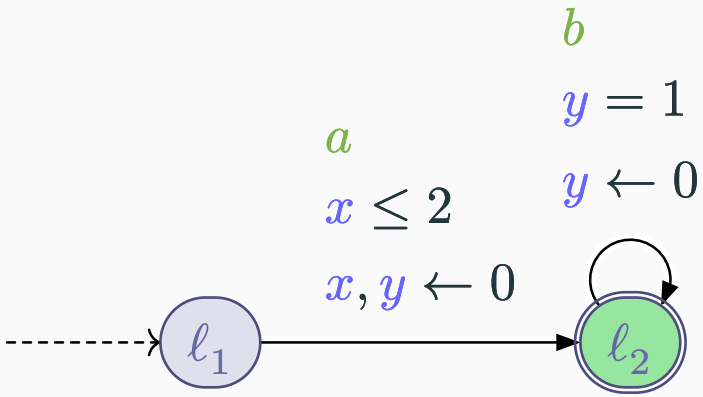


- $c_x = c_y = 1$
- We represent only the reachable nodes
- The transition $\ell_2 \to \ell_3$ cannot be taken



Exercise: Region graph construction

Construct the region graph of the following timed automaton



Region graph

- Is the region graph of timed automata always finite? ✓

Complexity

- The region graph is **exponential** in the number of clocks
- However, it remains **finite**, which enables model checking algorithms

Zones

Symbolic states: zones

Objective: group all concrete states reachable by the same sequence of discrete actions

Definition 1 (Symbolic state)

A **symbolic state** is a pair (ℓ, Z) where:

- ℓ is a location of the timed automaton
- Z is an (infinite) set of clock valuations

Symbolic states: zones

Zone representation

For timed automata, Z can be represented by a convex polyhedron with a special form called **zone**, with constraints of the form:

- $-d_0^i \leq x_i \leq d_1^i$
- $x_i - x_j \leq d_{i,j}$

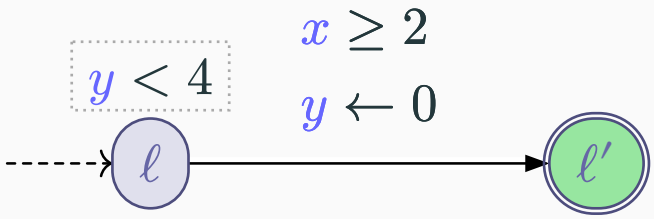
Symbolic computation

Computation of successive reachable symbolic states can be performed symbolically with polyhedral operations:

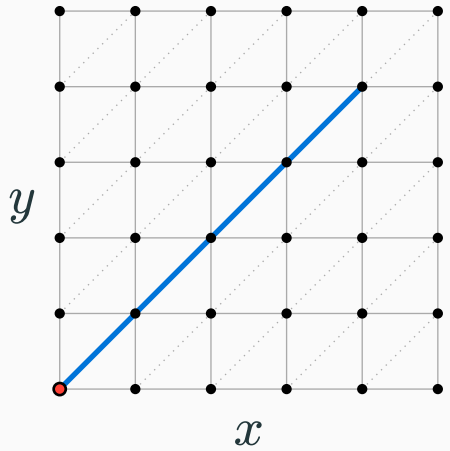
For edge $e = (\ell, a, g, R, \ell')$:

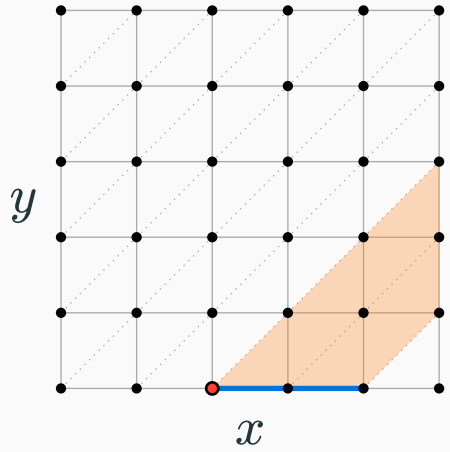
$$\mathrm{Succ}((\ell, Z), e) = \left(\ell', ([Z \cap g]_R \cap I(\ell'))^{\nearrow} \cap I(\ell')\right)$$

Example: Zones of a TA



$$\mathrm{Succ}((\ell, Z), e) = (\ell', [Z \cap g]_R \cap I(\ell'))^{\nearrow} \cap I(\ell'))$$



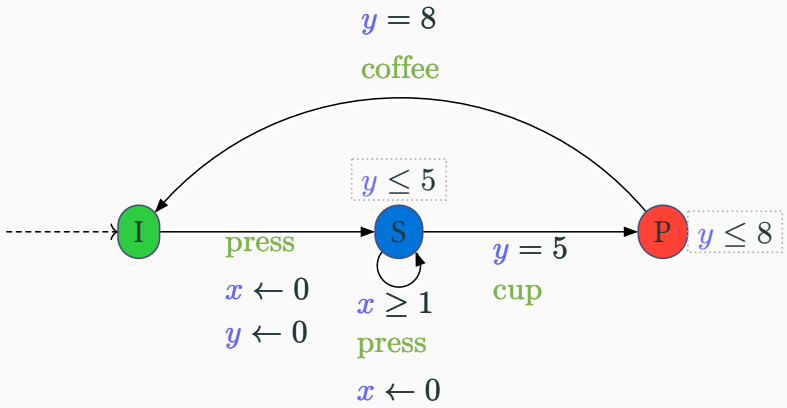


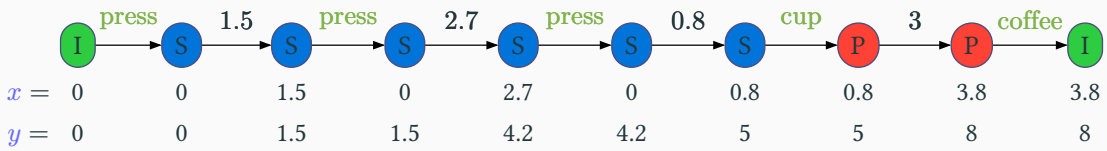
$$Z_0 = \{(0,0)\}^{\nearrow} \cap I(\ell)$$

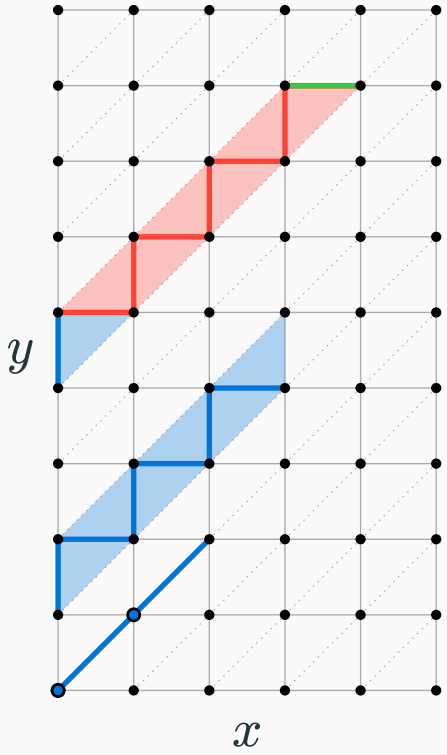
$$Z_1 = [Z_0 \cap (x \geq 2)]^{\nearrow}_{\{y\}} \cap \top$$

Exercise: Zone graph of a run

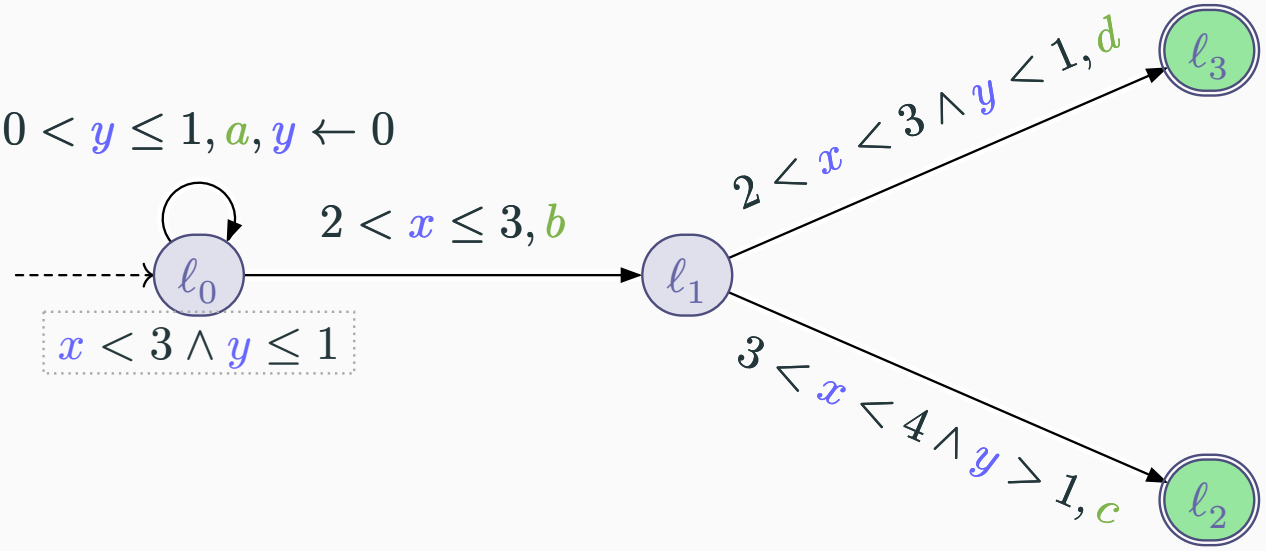
Depicted the zones reached along the following run in the timed coffee machine:







Consider the following timed automaton:



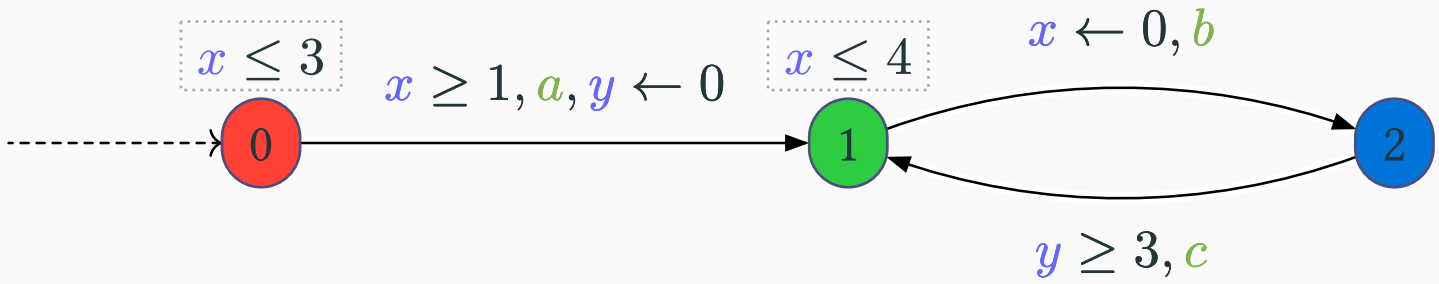
1. Give a run of this automaton that reaches each of the final states.
2. Represent in the plane the successive regions browsed by the clocks during these executions.

Solution: Exercise

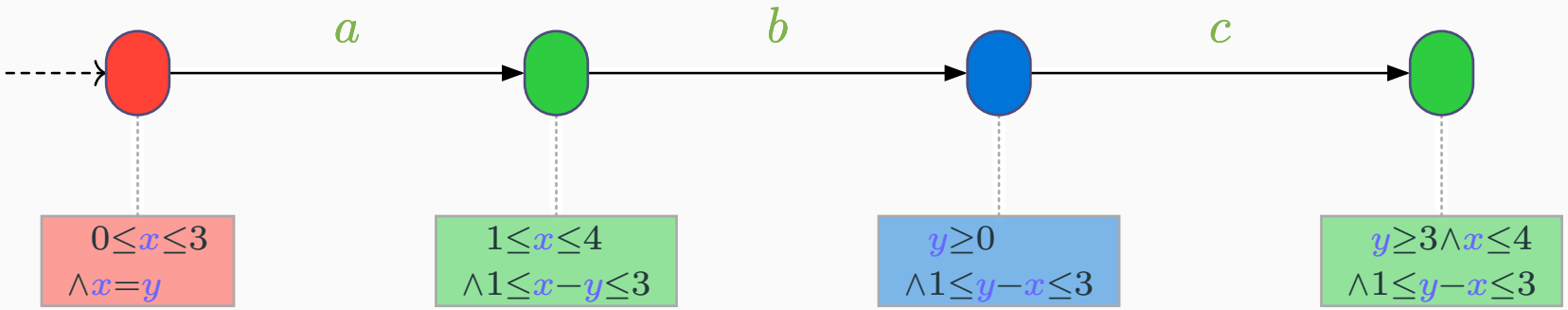
Abstract semantics of timed automata

- **Abstract state**: pair (ℓ, C) where:
  - $\ell \in L$ is a location
  - C is a constraint on the clocks (a **zone**)
- **Abstract run**: alternating sequence of abstract states and actions

Example: Abstract semantics



- A possible abstract run:



Timed temporal logics

MTL / MITL

Metric Temporal Logic (MTL)

Metric Temporal Logic (MTL)

- Extension of LTL with **timing constraints** on modalities
- Specify properties on the order and the **delay** between atomic propositions
- No X modality (dense time)

Definition 1 (Syntax of MTL)

$$\mathrm{MTL} \ni \varphi ::= p \mid \neg\varphi \mid \varphi \vee \varphi \mid \varphi \mathrm{U}_I \psi$$

with I an interval with bounds in $\mathbb{Q}^+ \cup \{\infty\}$

MTL: Extended syntax

As for LTL, we can define additional operators:

$\varphi \wedge \psi$	
$\varphi \implies \psi$	
$\varphi \iff \psi$	
$\mathrm{F}_I \varphi$	
$\mathrm{G}_I \varphi$	

MTL: Continuous semantics

Definition 2 (Semantics of MTL)

$\rho, t \models p$	if $p \in \mathrm{lab}(\rho(\ell))$
$\rho, t \models \neg\varphi$	if not $\rho, t \models \varphi$
$\rho, t \models \varphi \vee \psi$	if $\rho, t \models \varphi$ or $\rho, t \models \psi$
$\rho, t \models \varphi \mathrm{U}_{\sim c} \psi$	$\exists u$ s.t. $u > 0 : \rho, t + u \models \psi \wedge \forall 0 < v < u : \rho, t + v \models \varphi \wedge u \in I$

♣

Exercise: Writing MTL formulae

I will eventually get a job within a year	
The plane will never crash within the 8 hours of the flight	
Every time I ask a question, the teacher will answer me immediately	
During the next 4 hours, I will starve unless I get some food or drinks	

MTL model checking

Theorem 3

The model-checking problem for MTL *(under continuous semantics)* is **undecidable** [AH93]

♡

[AH93] Rajeev Alur and Thomas A. Henzinger, "Real-Time Logics: Complexity and Expressiveness," *Information and Computation*, 1993.

Metric Interval Temporal Logic (MITL)

Metric Interval Temporal Logic (MITL)

Definition 4 (Syntax of MITL [AFH96])

$\text{MTL} \ni \varphi ::= p \mid \neg\varphi \mid \varphi \vee \varphi \mid \varphi \mathrm{U}_I \psi$

with I a **non-punctual** interval with bounds in $\mathbb{Q}^+ \cup \{\infty\}$

♣

Theorem 5

The model-checking problem for MITL is **decidable** and EXPSACE-complete [AFH96]

♡

[AFH96] Rajeev Alur, Tomás Feder, and Thomas A. Henzinger, "The Benefits of Relaxing Punctuality," *Journal of the ACM*, 1996.

Exercise: MITL formulae

The plane will never crash within the 8 hours of the flight	
I will get a job in one year	
During the next 4 hours, I will starve unless I get some food or drinks	
Every time I ask a question, the teacher will answer me at least 2 and at most 4 minutes later	
Every time I ask a question, the teacher will answer me immediately	

Methods for MITL model checking

Checking whether $\mathcal{A} \models \varphi$

Similar to LTL:

- Construct the TA $\mathcal{B}_{\neg\varphi}$ recognizing all executions not satisfying φ [AFH96]
- Construct the synchronized product $\mathcal{A} \times \mathcal{B}_{\neg\varphi}$
- Check the emptiness of its timed language

Note: translating an MITL formula to a TA isn't trivial!

[AFH96] Rajeev Alur, Tomás Feder, and Thomas A. Henzinger, "The Benefits of Relaxing Punctuality," *Journal of the ACM*, 1996.

TCTL

TCTL: Timed CTL

[ACD93]

TCTL expresses formulas on the **order** and the **time** between the future atomic propositions **for some or for all** paths, over a set of atomic propositions

Definition 1 (Syntax of TCTL)

$$\text{TCTL} \ni \varphi ::= p \mid \neg\varphi \mid \varphi \vee \varphi \mid \mathrm{E}\ \varphi \mathrm{U}_{\sim c} \psi \mid \mathrm{A}\ \varphi \mathrm{U}_{\sim c} \psi$$

with $\sim \in \{<, \leq, =, \geq, >\}$ and $c \in \mathbb{Q}^+$

♣

- No X modality because of dense time

[ACD93] Rajeev Alur, Costas Courcoubetis, and David L. Dill, "Model-Checking in Dense Real-time," *Information and Computation*, 1993.

TCTL: Semantics

Definition 2 (Semantics of TCTL)

$(\ell, \mu) \models p$	if $p \in \mathrm{lab}(\ell)$
$(\ell, \mu) \models \neg\varphi$	if not $(\ell, \mu) \models \varphi$
$(\ell, \mu) \models \varphi \vee \psi$	if $(\ell, \mu) \models \varphi$ or $(\ell, \mu) \models \psi$
$(\ell, \mu) \models \mathrm{E}\ \varphi \mathrm{U}_{\sim c} \psi$	if there is a run from (ℓ, μ) to (ℓ', μ') *s.t.* ▸ $t(\mu') - t(\mu) \sim c$ ▸ for all (ℓ'', μ'') between (ℓ, μ) and (ℓ', μ'), $(\ell'', \mu'') \models \varphi$ ▸ $(\ell', \mu') \models \psi$
$(\ell, \mu) \models \mathrm{A}\ \varphi \mathrm{U}_{\sim c} \psi$	if for all runs ...

TCTL: Extended syntax

As for CTL, we can define additional operators:

$\varphi \wedge \psi$	
$\varphi \Longrightarrow \psi$	
$\varphi \Longleftrightarrow \psi$	
$\mathrm{E}\,\mathrm{F}_I \varphi$	
$\mathrm{A}\,\mathrm{F}_I \varphi$	
$\mathrm{E}\,\mathrm{G}_I \varphi$	
$\mathrm{A}\,\mathrm{G}_I \varphi$	

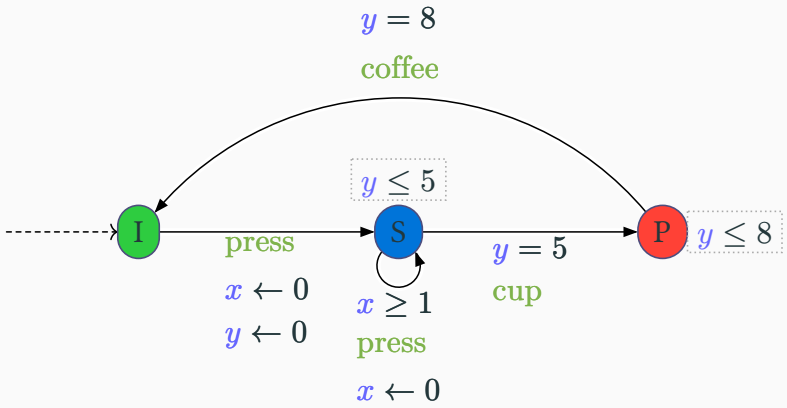
Exercise: TCTL formulae

Whatever happens, the plane will never crash in the next 10 minutes	
I may get a job in one year	
Whenever a fire breaks, it is sure that the alarm will start ringing at least 5 seconds and at most 10 seconds later	
Whatever happens, I will love you for 2 years after we marry	

Exercise: Coffee machine

Express in TCTL the following properties, and decide whether they are satisfied for the coffee machine

We reason here with actions instead of state predicate



- "Whenever the button is pressed, a coffee is necessarily eventually delivered within 10 units of time"
- "It must never happen that the button can be pressed twice within 1 unit of time"
- It must never happen that the button can be pressed twice within a time strictly less than 1 unit of time

TCTL model checking

Lemma 3 (Region equivalence)

Let ℓ be a location and prop a TCTL formula. For any two equivalent clock valuations $\mu \approx \mu'$ (belonging to the **same region**), we have:

$$(\ell, \mu) \vDash \varphi \iff (\ell, \mu') \vDash \varphi$$

♡

Theorem 4 (Decidability)

The model-checking problem for TCTL is **decidable** and PSPACE-complete [ACD93]

♡

[ACD93] Rajeev Alur, Costas Courcoubetis, and David L. Dill, "Model-Checking in Dense Real-time," *Information and Computation*, 1993.

Observers

Using observers

Observers are (timed) automata used to monitor system behavior and verify properties

Observer characteristics

An observer is a timed automaton that:

- **Synchronizes** with the system on shared actions
- Can use its **own clocks** to measure time constraints
- Can **read clocks** of the system
- Must be **non-blocking** (no timelocks or deadlocks)
- Has locations indicating whether a property holds

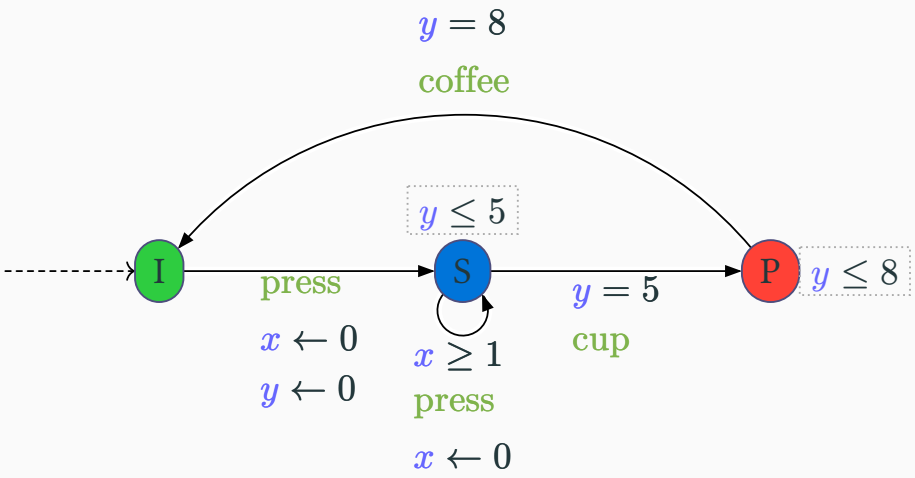
Reduction to reachability

Verifying a property φ can be reduced to a **reachability problem** in the product automaton $\mathcal{A} \times \mathcal{O}_\varphi$

where $\mathcal{O}_\varphi$ is an observer for property φ

This reduction allows us to use existing reachability algorithms for model checking temporal properties!

Exercise: Designing an observer



- Design an observer for the coffee machine verifying that it must never happen that the button can be pressed twice within a time strictly less than 1 unit of time.
- Express the reachability property to be checked

Solution: Designing an observer

Open note

In practice

Tools

- HyTech *(HA, PTA)* [HHW97]
- Kronos [Yov97]
- TReX *(PTA)* [ABS01]
- Uppaal [LPY97]
- Roméo *(PT Petri nets)* [Lim+09]
- PAT *(multiple formalisms)* [Sun+09]
- IMITATOR *(PTA)* [And21]

Case studies

- **Real-time systems** & scheduling [AM12], [AAM06], [AM01], [Feh99]
- Protocols [SS01], [TY98], [D'A+97], [Hav+97]
- **Hardware** circuits [Che+09], [Boz+02]
- **Health** & biology [Sch+14]
- **Monitoring** [WHS18], [WAH16]
- Uppaal industrial survey [LLN18]

Timed automata
Syntax
Semantics
Specifying with timed automata

Some words about decidablity
Regions
Zones

Timed temporal logics
MTL / MITL
TCTL
Observers

Open note

Bibliography

Bibliography

[AD94] Rajeev Alur and David L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, 1994.

[Mer74] Philip Meir Merlin, "A study of the recoverability of computing systems," Doctoral dissertation, 1974.

[Sun+09] Jun Sun, Yang Liu, Jin Song Dong, and Xian Zhang, "Verifying Stateful Timed CSP Using Implicit Clocks and Zone Abstraction," in *ICFEM 2009*, 2009.

[Bér+05] Béatrice Bérard, Franck Cassez, Serge Haddad, Didier Lime, and Olivier H. Roux, "Comparison of the Expressiveness of Timed Automata and Time Petri Nets," 2005.

[Srb08] Jirí Srba, "Comparing the Expressiveness of Timed Automata and Timed Extensions of Petri Nets," in *FORMATS 2008*, 2008.

[Bér+13] Béatrice Bérard, Franck Cassez, Serge Haddad, Didier Lime, and Olivier H. Roux, "The expressive power of time Petri nets," *Theoretical Computer Science*, 2013.

[Hen+94] Thomas A. Henzinger, Xavier Nicollin, Joseph Sifakis, and Sergio Yovine, "Symbolic Model Checking for Real-Time Systems," *Information and Computation*, 1994.

[HKW95] Thomas A. Henzinger, Peter W. Kopke, and Howard Wong-Toi, "The Expressive Power of Clocks," 1995.

[AHV93] Rajeev Alur, Thomas A. Henzinger, and Moshe Y. Vardi, "Parametric real-time reasoning," 1993.

[Tri06] Stavros Tripakis, "Folk theorems on the determinization and minimization of timed automata," *Information Processing Letters*, 2006.

Bibliography

[Fin06] Olivier Finkel, "Undecidable Problems About Timed Automata," in *FORMATS 2006*, 2006.

[BY03] Johan Bengtsson and Wang Yi, "Timed Automata: Semantics, Algorithms and Tools," 2003.

[Bou08] Patricia Bouyer, "Timed Automata," 2008.

[AH93] Rajeev Alur and Thomas A. Henzinger, "Real-Time Logics: Complexity and Expressiveness," *Information and Computation*, 1993.

[AFH96] Rajeev Alur, Tomás Feder, and Thomas A. Henzinger, "The Benefits of Relaxing Punctuality," *Journal of the ACM*, 1996.

[ACD93] Rajeev Alur, Costas Courcoubetis, and David L. Dill, "Model-Checking in Dense Real-time," *Information and Computation*, 1993.

[HHW97] Thomas A. Henzinger, Pei-Hsin Ho, and Howard Wong-Toi, "HYTECH: A Model Checker for Hybrid Systems," *Int. J. Softw. Tools Technol. Transf.*, 1997.

[Yov97] Sergio Yovine, "KRONOS: A Verification Tool for Real-Time Systems," *International Journal on Software Tools for Technology Transfer*, 1997.

[ABS01] Aurore Annichini, Ahmed Bouajjani, and Mihaela Sighireanu, "TReX: A Tool for Reachability Analysis of Complex Systems," in *CAV 2001*, 2001.

Bibliography

[LPY97] Kim Guldstrand Larsen, Paul Pettersson, and Wang Yi, "UPPAAL in a Nutshell," *International Journal on Software Tools for Technology Transfer*, 1997.

[Lim+09] Didier Lime, Olivier H. Roux, Charlotte Seidner, and Louis-Marie Traonouez, "Romeo: A Parametric Model-Checker for Petri Nets with Stopwatches," in *TACAS*, 2009.

[Sun+09] Jun Sun, Yang Liu, Jin Song Dong, and Jun Pang, "PAT: Towards Flexible Verification under Fairness," 2009.

[And21] Étienne André, "IMITATOR~3: Synthesis of timing parameters beyond decidability," in *CAV*, 2021.

[Feh99] Ansgar Fehnker, "Scheduling a Steel Plant with Timed Automata," in *6th International Workshop on Real-Time Computing and Applications Symposium (RTCSA '99), 13-16 December 1999, Hong Kong, China*, 1999.

[AM01] Yasmina Abdeddaïm and Oded Maler, "Job-Shop Scheduling Using Timed Automata," in *CAV 2001*, 2001.

[AAM06] Yasmina Abdeddaïm, Eugene Asarin, and Oded Maler, "Scheduling with timed automata," *Theoretical Computer Science*, 2006.

[AM12] Yasmina Abdeddaïm and Damien Masson, "Real-Time Scheduling of Energy Harvesting Embedded Systems with Timed Automata," in *RTCSA 2012*, 2012.

[D'A+97] Pedro R. D'Argenio, Joost-Pieter Katoen, Theo C. Ruys, and Jan Tretmans, "The Bounded Retransmission Protocol Must Be on Time!," in *Tools and Algorithms for Construction and Analysis of Systems, Third International Workshop, TACAS '97, Enschede, The Netherlands, April 2-4, 1997, Proceedings*, 1997.

Bibliography

[Hav+97] Klaus Havelund, Arne Skou, Kim Guldstrand Larsen, and Kristian Lund, "Formal modeling and analysis of an audio/video protocol: an industrial case study using UPPAAL," in *Proceedings of the 18th IEEE Real-Time Systems Symposium (RTSS '97), December 3-5, 1997, San Francisco, CA, USA*, 1997.

[TY98] Stavros Tripakis and Sergio Yovine, "Verification of the Fast Reservation Protocol with Delayed Transmission using the Tool Kronos," in *Proceedings of the Fourth IEEE Real-Time Technology and Applications Symposium, RTAS'98, Denver, Colorado, USA, June 3-5, 1998*, 1998.

[SS01] David P. L. Simons and Mariëlle Stoelinga, "Mechanical verification of the IEEE 1394a root contention protocol using Uppaal2k," *Int. J. Softw. Tools Technol. Transf.*, 2001.

[Boz+02] Marius Bozga, Jianmin Hou, Oded Maler, and Sergio Yovine, "Verification of Asynchronous Circuits using Timed Automata," in *Theory and Practice of Timed Systems, Satellite Event of ETAPS 2002, Grenoble, France, April 6-7, 2002*, 2002.

[Che+09] Remy Chevallier, Emmanuelle Encrenaz-Tiphène, Laurent Fribourg, and Weiwen Xu, "Timed verification of the generic architecture of a memory circuit using parametric timed automata," *Formal Methods Syst. Des.*, 2009.

[Sch+14] Stefano Schivo *et al.*, "Modeling Biological Pathway Dynamics With Timed Automata," *IEEE J. Biomed. Health Informatics*, 2014.

[WAH16] Masaki Waga, Takumi Akazaki, and Ichiro Hasuo, "A Boyer-Moore Type Algorithm for Timed Pattern Matching," in *FORMATS 2016*, 2016.

Bibliography

[WHS18] Masaki Waga, Ichiro Hasuo, and Kohei Suenaga, "MONAA: A Tool for Timed Pattern Matching with Automata-Based Acceleration," in *3rd Workshop on Monitoring and Testing of Cyber-Physical Systems, MT@CPSWeek 2018, Porto, Portugal, April 10, 2018*, 2018.

[LLN18] Kim G. Larsen, Florian Lorber, and Brian Nielsen, "20 Years of UPPAAL Enabled Industrial Model-Based Validation and Beyond," in *Leveraging Applications of Formal Methods, Verification and Validation. Industrial Practice - 8th International Symposium, ISoLA 2018, Limassol, Cyprus, November 5-9, 2018, Proceedings, Part IV*, 2018.

Timed automata
Syntax
Semantics
Specifying with timed automata

Some words about decidablity
Regions
Zones

Timed temporal logics
MTL / MITL
TCTL
Observers

Open note

Additional information

License of this document

Author: **Dylan Marinho**

Inspired by the courses of Étienne André

SCIENCES SORBONNE UNIVERSITÉ

Version: 2025-12-18